

Obiettivo

Lo scopo principale di questo progetto è creare una CNN che provi ad identificare il "tipo" di sfocatura applicata all'immagine, dove con "tipo" si intende il kernel con cui è stata convoluta l'immagine. Nel dettaglio sono stati presi in considerazione i seguenti kernel:

UNIFORMI

Box_3

-			
	0.1111	0.1111	0.1111
	0.1111	0.1111	0.1111
	0.1111	0.1111	0.1111

Box_5

-				
0.0400	0.0400	0.0400	0.0400	0.0400
0.0400	0.0400	0.0400	0.0400	0.0400
0.0400	0.0400	0.0400	0.0400	0.0400
0.0400	0.0400	0.0400	0.0400	0.0400
0.0400	0.0400	0.0400	0.0400	0.0400

GAUSSIANI

Gauss_3

-		
0.0625	0.1250	0.0625
0.1250	0.2500	0.1250
0.0625	0.1250	0.0625

Gauss_5

-				
0.0039	0.0156	0.0234	0.0156	0.0039
0.0156	0.0625	0.0938	0.0625	0.0156
0.0234	0.0938	0.1406	0.0938	0.0234
0.0156	0.0625	0.0938	0.0625	0.0156
0.0039	0.0156	0.0234	0.0156	0.0039

FORME

Circ_5

-				
0	0	0.0769	0	0
0	0.0769	0.0769	0.0769	0
0.0769	0.0769	0.0769	0.0769	0.0769

0	0.0769	0.0769	0.0769	0
0	0	0.0769	0	0

-

Cross_5

-

0	0	0.1111	0	0
0	0	0.1111	0	0
0.1111	0.1111	0.1111	0.1111	0.1111
0	0	0.1111	0	0
0	0	0.1111	0	0

-

LINEE

D

-

0.2000	0	0	0	0
0	0.2000	0	0	0
0	0	0.2000	0	0
0	0	0	0.2000	0
0	0	0	0	0.2000

-

H

-

0	0	0	0	0
0	0	0	0	0
0.2000	0.2000	0.2000	0.2000	0.2000
0	0	0	0	0
0	0	0	0	0

-

V

-

0	0	0.2000	0	0
0	0	0.2000	0	0
0	0	0.2000	0	0
0	0	0.2000	0	0
0	0	0.2000	0	0

-

ALTRI

Originale

-

0	0	0
0	1	0
0	0	0

-

Median_5: restituisce la mediana della finestra

Il dataset usato è "[COCO 2017 Unlabeled Images](#)". Questa scelta deriva dall'obiettivo generale, infatti non si vuole classificare un qualcosa presente nell'immagine ma una caratteristica di quest'ultima e quindi sorge la necessità di avere un dataset molto vario per permettere una buona generalizzazione. Per capire appunto la capacità di generalizzazione è stato creato un secondo dataset di test (combinando "[Sea Animals Image Dataset](#)" e "[Count Coins Image dataset](#)") da usare in parallelo allo split apposito del dataset principale.

In ogni caso tutti i dataset erano composti da immagini più o meno nitide di dimensioni non necessariamente fisse, per adattarle alla rete usata è stato quindi necessario fare delle operazioni di pre-processing. Le 2 principali sono:

1. Togliere le immagini già sfocate in partenza per evitare problematiche varie (seguendo "[Blur detection with OpenCV](#)")
2. Effettuare la convoluzione e ridimensionare il risultato

Per ogni singola immagine si può riassumere il flusso di elaborazione come:

1. Apri immagine in scala di grigi
2. Se troppo piccola/sfocata ignorala
3. Applica la convoluzione con un kernel casuale (la grandezza del dataset garantisce una distribuzione abbastanza uniforme)
4. Ritaglia una porzione di dimensione 224x224 al centro (per evitare di includere eventuali artefatti sui bordi)
5. Salva risultato

Dopo aver completato l'elaborazione si è arrivati a questi 2 dataset:

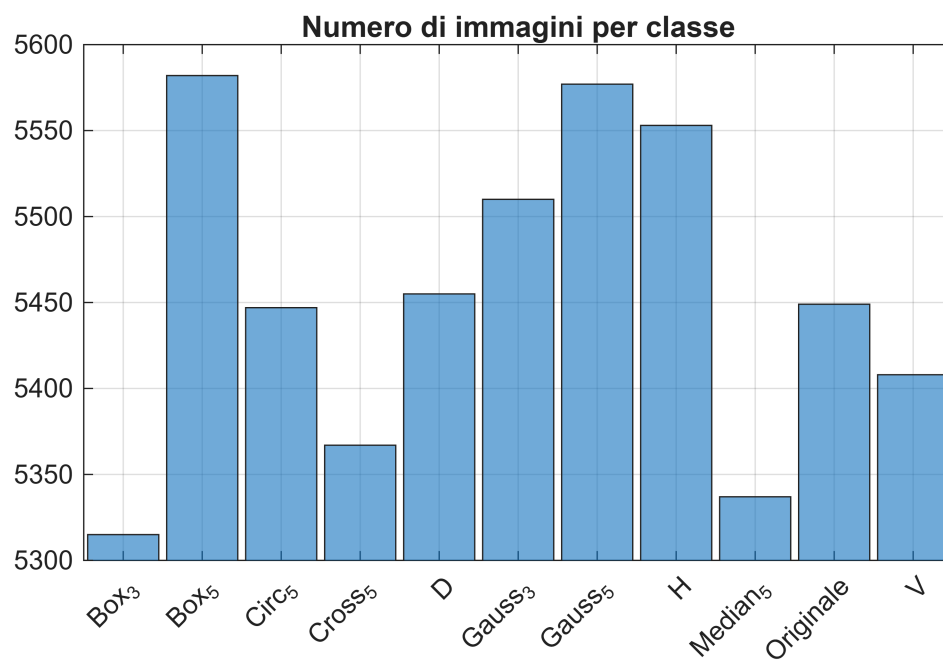
DATASET PRINCIPALE

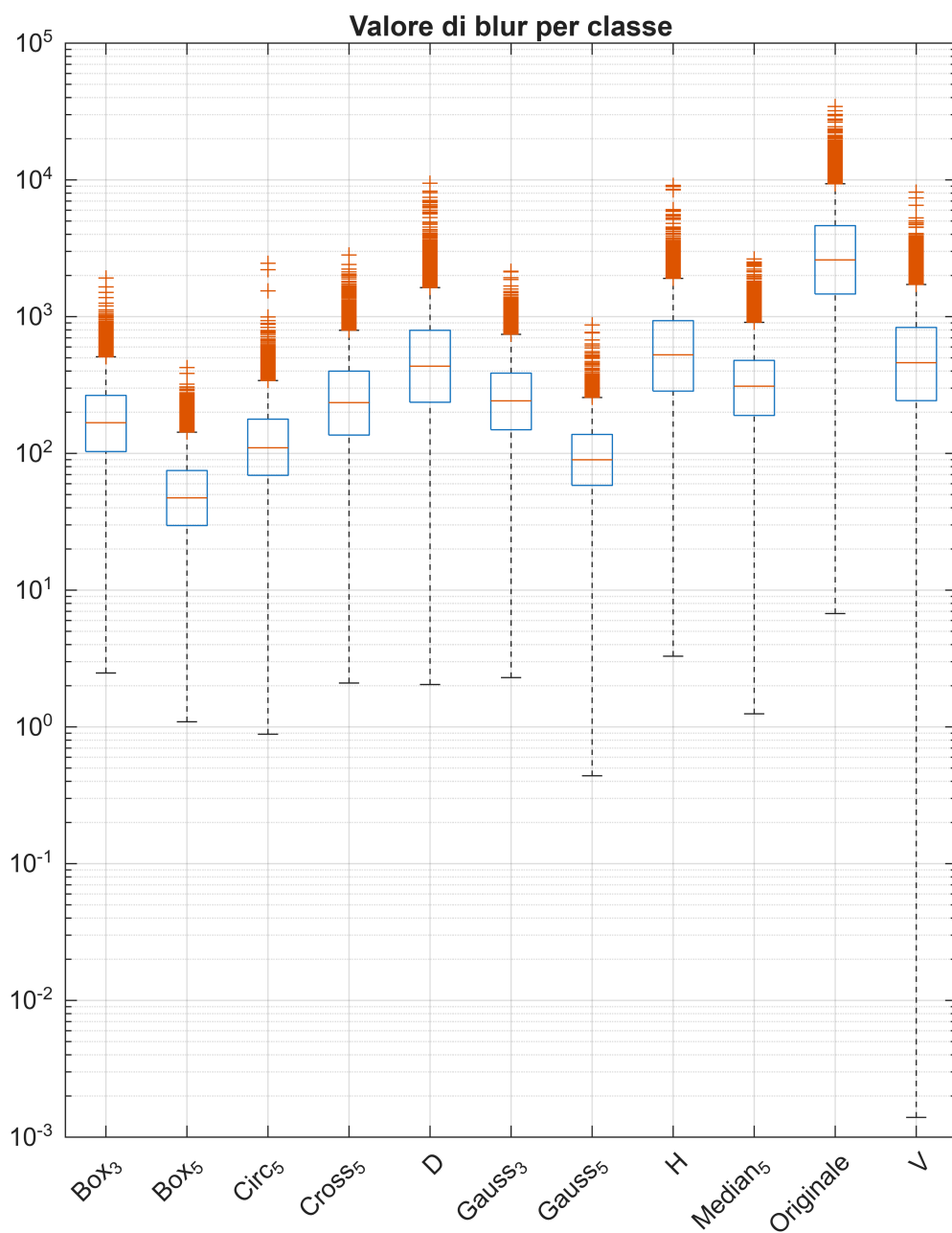
Numero totale di immagini: 60000

Training: 42000 (70%)

Validazione: 9000 (15%)

Test: 9000 (15%)

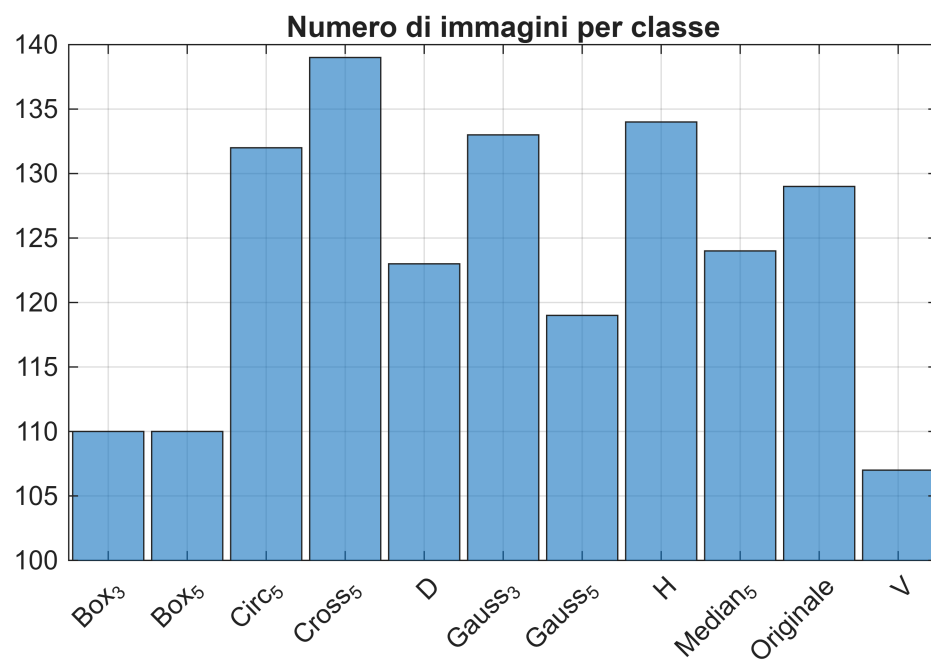


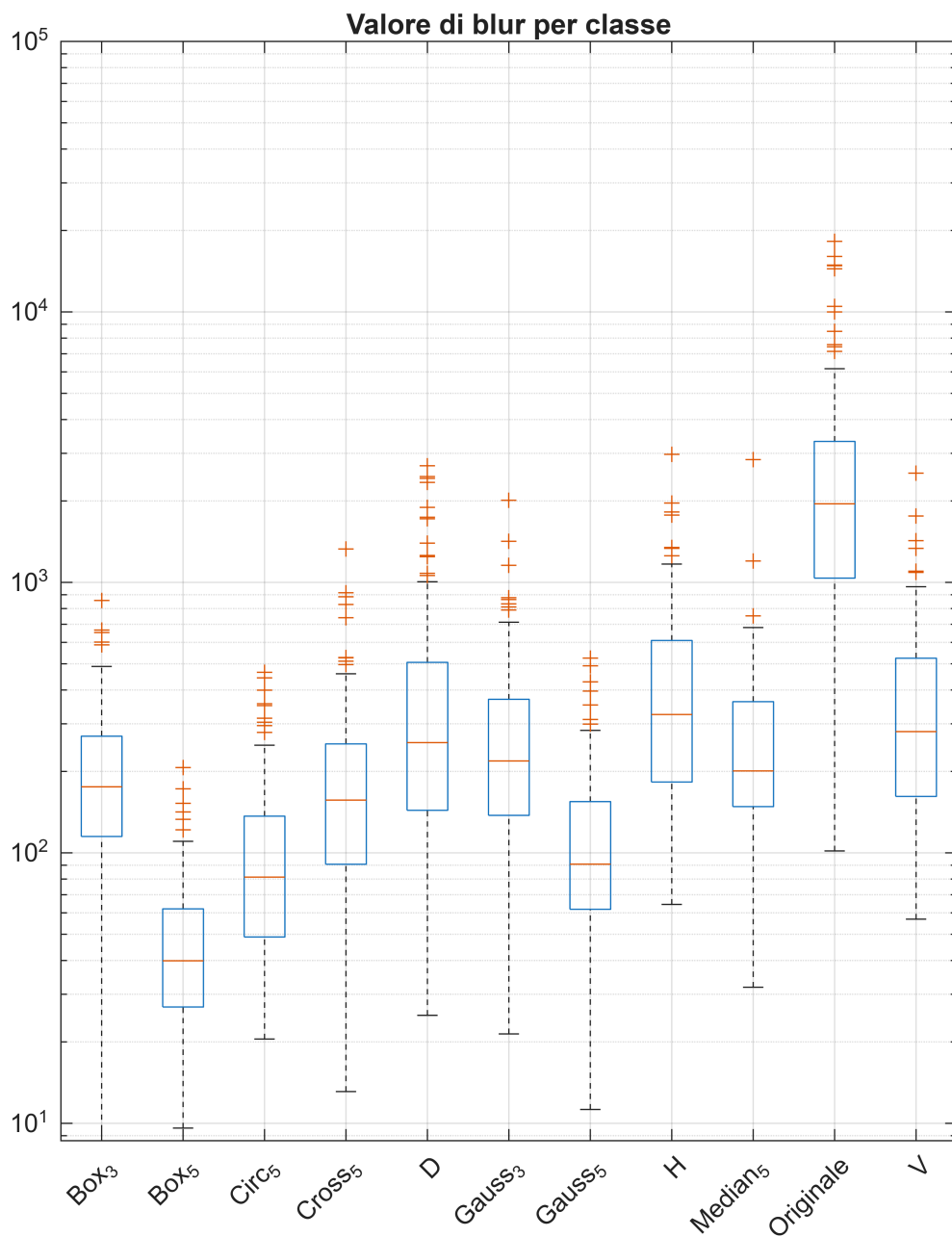


I valori sull'asse delle ordinate sono calcolati con il metodo citato prima per trovare le immagini sfocate, però dopo la convoluzione.

DATASET CONTRO-TEST

Numero totale di immagini: 1360





Struttura e addestramento della rete

La rete finale presenta la seguente struttura:

```
-----
Layer 1: Input
Input: [224 224 1]
```

```
-----
Layer 2: Convoluzione
Grandezza filtro: [5 5]
Numero filtri: 8
Stride: [1 1]
-----
```

```

Layer 3: Normalizzazione batch
-----
Layer 4: ReLU
-----
Layer 5: Convoluzione
Grandezza filtro: [3 3]
Numero filtri: 16
Stride: [1 1]
-----
Layer 6: Normalizzazione batch
-----
Layer 7: ReLU
-----
Layer 8: Max Pooling
Grandezza pooling: [2 2]
Stride: [2 2]
-----
Layer 9: Convoluzione
Grandezza filtro: [3 3]
Numero filtri: 32
Stride: [1 1]
-----
Layer 10: Normalizzazione batch
-----
Layer 11: ReLU
-----
Layer 12: Max Pooling
Grandezza pooling: [2 2]
Stride: [2 2]
-----
Layer 13: Dropout
Probabilità: 30.00%
-----
Layer 14: Completamente connesso
Numero neuroni: 11
-----
Layer 15: Softmax
-----
Layer 16: Classoutput

```

E le seguenti opzioni di addestramento:

```

Ottimizzatore:          SGDM
Learning rate iniziale:  3.0e-04
Learn. rate schedule:    piecewise
Learn. rate drop factor: 0.10
Learn. rate drop period: 15 epoche
Numero max epoche:       30
Dimensioni mini-batch:   32
Shuffle dati:             every-epoch
Frequenza di validazione: 438 iterazioni
Pazienza di validazione: 5
Ambiente di esecuzione:  cpu

```

Inoltre come misura aggiuntiva contro l'overfitting è stata utilizzata un'augmentation adatta al dataset:

1. Traslazione orizzontale ± 5 px
2. Traslazione verticale ± 5 px
3. Zoom $\pm 5\%$

Nel lungo processo che ha portato al modello finale sono emersi degli elementi che, per questo specifico compito, tendevano ad abbassare la performance generale del modello:

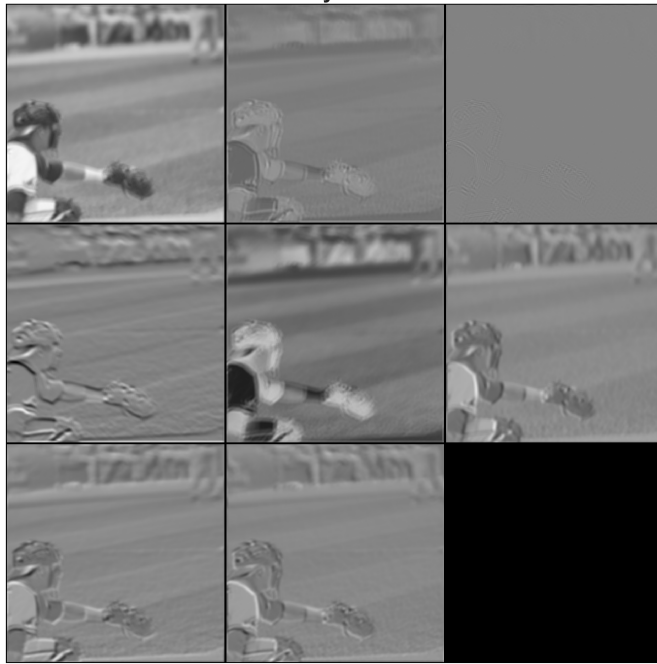
- Troppi strati convoluzionali
- Layer convoluzionali con troppi filtri o con filtri troppo grandi
- Average/Min Pooling
- Global Pooling
- Funzioni di attivazione diverse dalla ReLU
- ADAM

Esempio di funzionamento

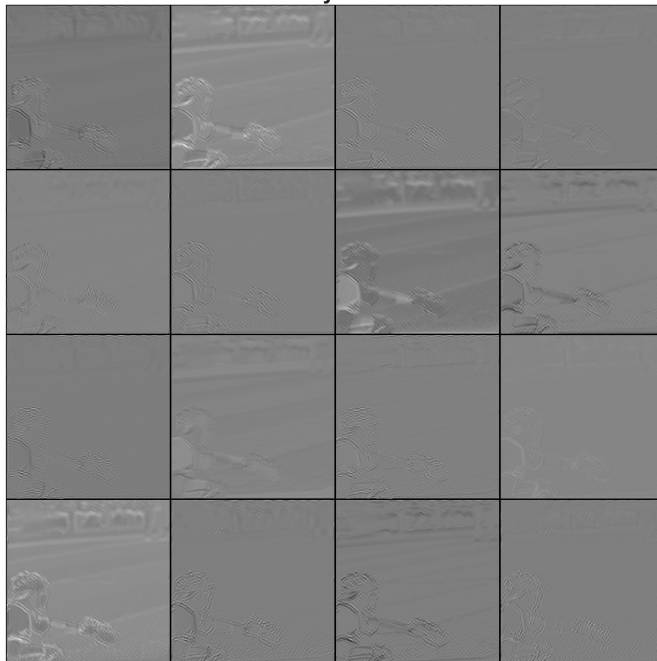
Input (Box₃)



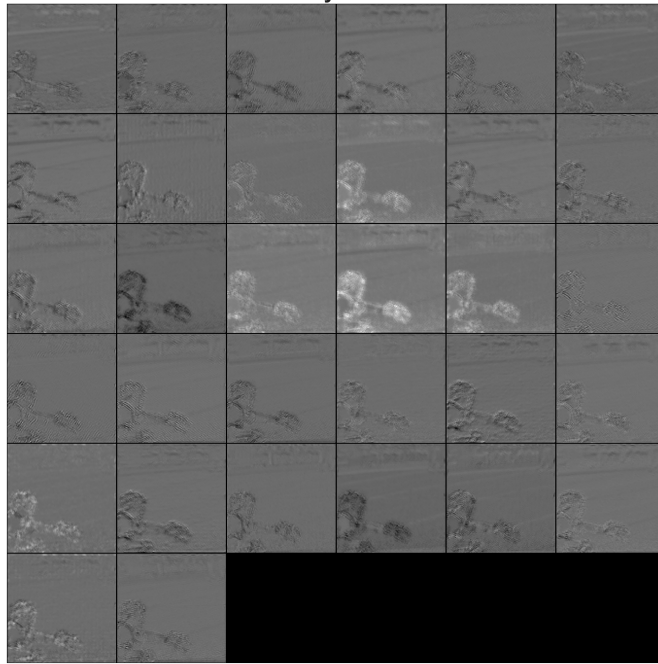
Layer 2



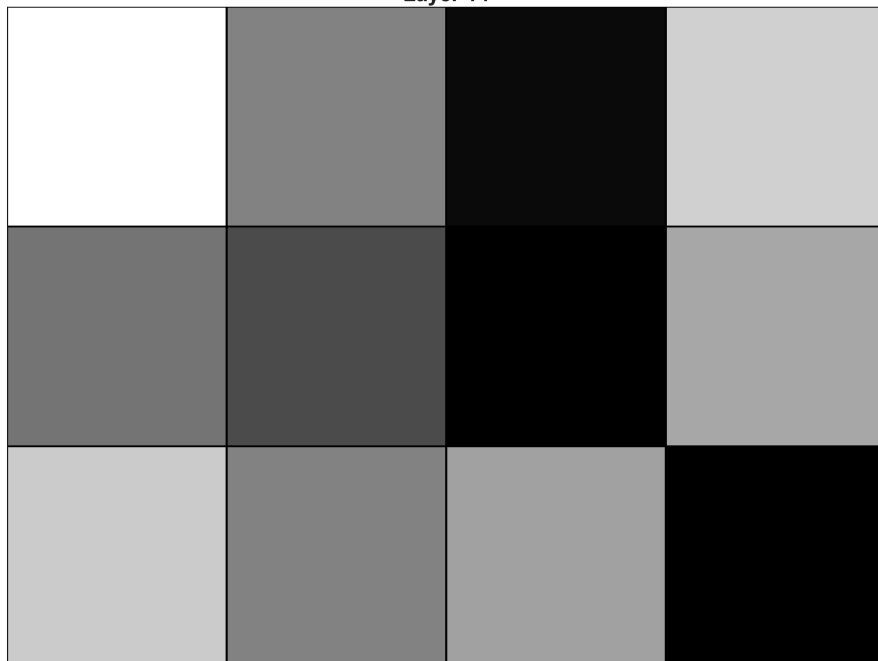
Layer 5



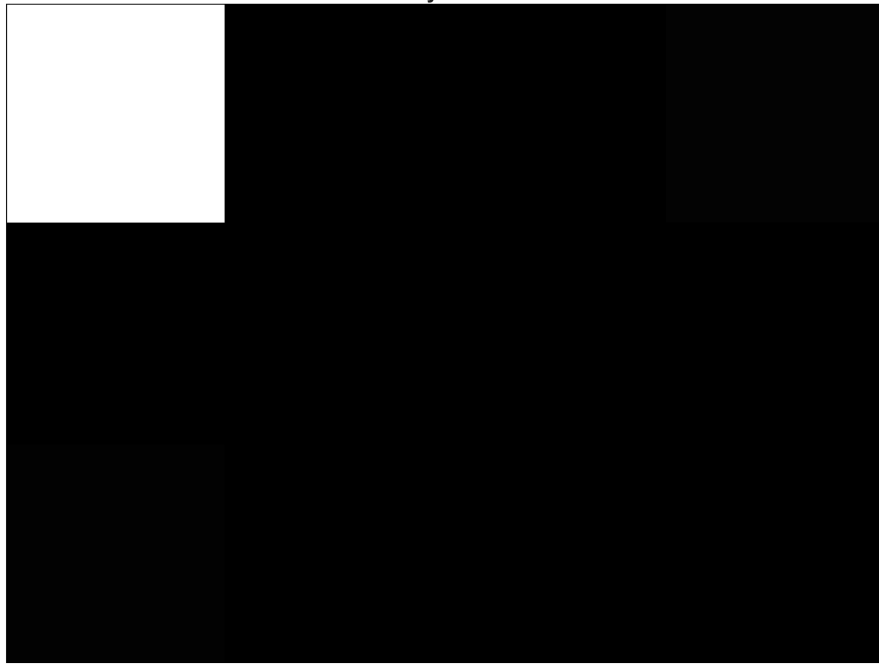
Layer 9



Layer 14



Layer 15



Predetta: Box_3 | CORRETTA

Risultati e metriche

DATASET PRINCIPALE

Matrice di confusione (Accuratezza = 91.81%)														
Classe reale	Box ₃	765	1		15	2	1	1		9	1	2	96.0%	4.0%
	Box ₅		803		10				11	2		11	95.9%	4.1%
	Circ ₅	1	1	793	14	4				4			97.1%	2.9%
	Cross ₅	1	1	1	793				1	6	2		98.5%	1.5%
	D			8	1	804				3	2		98.3%	1.7%
	Gauss ₃		1	2		1	798	1	5	2	14	2	96.6%	3.4%
	Gauss ₅	135	8	7			285	401					48.0%	52.0%
	H	1	3		6				800	15	8		96.0%	4.0%
	Median ₅		1							795	2	2	99.4%	0.6%
	Originale				1	1				96	719		88.0%	12.0%
V			1	9					12		789	97.3%	2.7%	

84.7%	98.0%	97.7%	93.4%	99.0%	73.6%	99.5%	97.9%	84.2%	96.1%	97.9%
15.3%	2.0%	2.3%	6.6%	1.0%	26.4%	0.5%	2.1%	15.8%	3.9%	2.1%
Box ₃	Box ₅	Circ ₅	Cross ₅	D	Gauss ₃	Gauss ₅	H	Median ₅	Originale	V

Classe predetta

Valori medi:

Recall: 91.92%

F1-Score: 91.34%

Valori per classe:

Classe	Specificità	F1-Score
{ 'Box_3' }	0.98317	0.9
{ 'Box_5' }	0.99804	0.96981
{ 'Circ_5' }	0.99768	0.9736
{ 'Cross_5' }	0.99316	0.95889
{ 'D' }	0.99902	0.9865
{ 'Gauss_3' }	0.965	0.8356
{ 'Gauss_5' }	0.99975	0.6473
{ 'H' }	0.99792	0.9697
{ 'Median_5' }	0.98182	0.9117
{ 'Originale' }	0.99645	0.91885
{ 'V' }	0.99792	0.97588

DATASET DI CONTRO-TEST

Matrice di confusione (Accuratezza = 91.62%)											
Box ₃	110										
Box ₅		105								5	
Circ ₅	2	2	127	1							
Cross ₅	2	2	1	130					1	2	1
D					123						
Gauss ₃	6					123	2			1	1
Gauss ₅	51		1			12	55				
H	1			2				127	1	3	
Median ₅									124		
Originale					1				12	116	
V									1		106

100.0%	
95.5%	4.5%
96.2%	3.8%
93.5%	6.5%
100.0%	
92.5%	7.5%
46.2%	53.8%
94.8%	5.2%
100.0%	
89.9%	10.1%
99.1%	0.9%

64.0%	96.3%	98.4%	97.7%	99.2%	91.1%	96.5%	100.0%	89.2%	95.1%	93.8%
36.0%	3.7%	1.6%	2.3%	0.8%	8.9%	3.5%		10.8%	4.9%	6.2%
Box ₃	Box ₅	Circ ₅	Cross ₅	D	Gauss ₃	Gauss ₅	H	Median ₅	Originale	V

Classe predetta

I valori tra i 2 dataset non risultano troppo distanti, quindi il modello riesce a generalizzare.

2 problemi

Guardando entrambe le matrici si può notare che il kernel Gaussiano 5x5 è quello che causa più problemi, difatti viene confuso molto spesso sia con la sua versione 3x3 che con Box 3x3. Per capire il perché basta confrontare delle immagini elaborate con questi kernel, facendolo si può constatare come ad occhio nudo i risultati risultino estremamente simili. Aggiungendo il fatto che tutte le varie architetture provate hanno presentato il medesimo problema, pur in misura diversa, si può dire con una certa confidenza che questa problematica derivi dal kernel in sé.

Un altro caso da considerare è quello della mediana, il fatto che la CNN classifichi alcune immagini inalterate con questa classe dipende dal funzionamento della mediana stessa. Se l'immagine a cui viene applicata è molto nitida (soprattutto con dei bordi ben definiti) si ha come effetto principale uno smussamento lungo i bordi mentre nel resto dell'immagine appare solamente una leggerissima attenuazione.

Originale:



Mediana 5x5:

Esempio

Box 3x3:

Esempio

Gauss 3x3:

Esempio

Gauss 5x5:

Esempio